

## Agenda

- How to approach **Schema Design** //
- How to code: **MVC** Architecture //
- **Design Tic Tac Toe.**

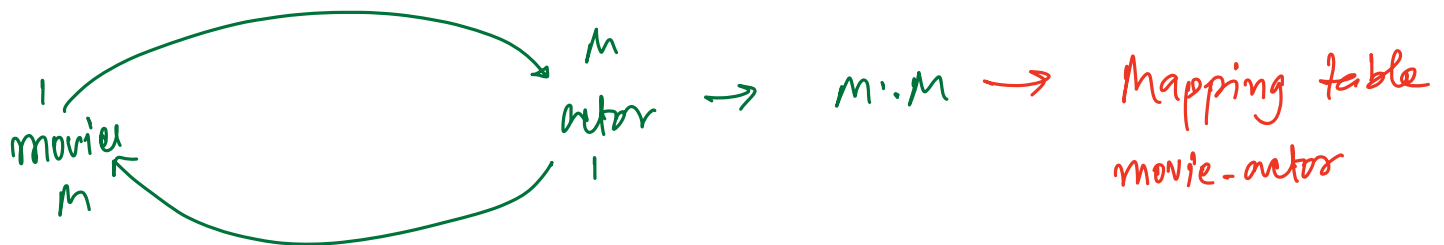
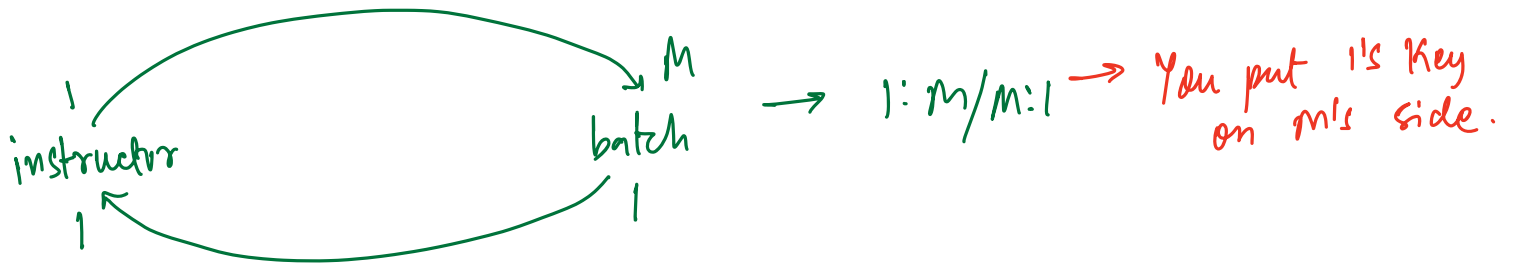
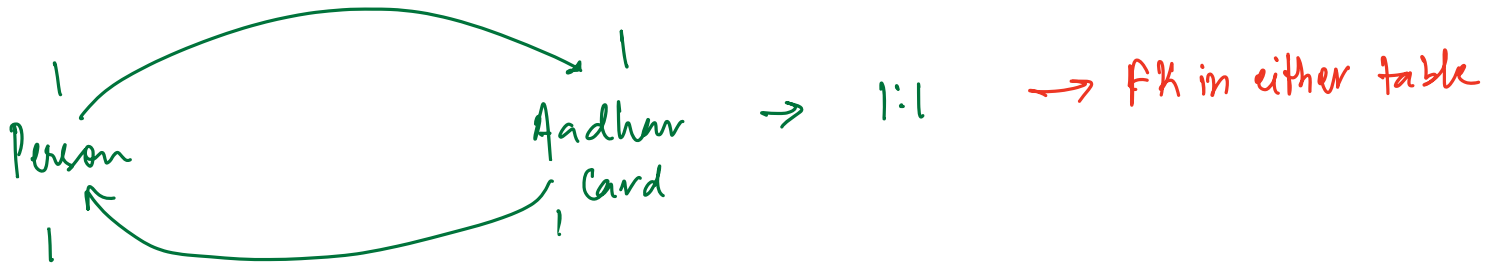
## Recap

- Step 1 → Get an overview
- Step 2 → Gather and clarify requirements
- Step 3 → Class Diagram ↷
- Step 4 → Schema Design (if required)
- Step 5 → Code end to end.

→ After class Diagram → Schema Design.

class Movie {  
 int id;  
 String name;  
 List<Actor> actors;  
}

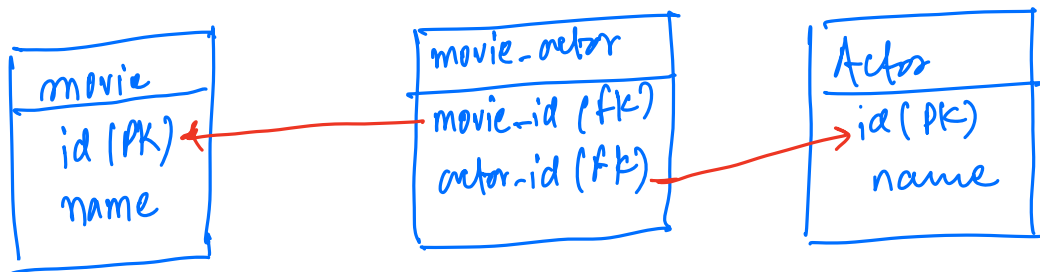
} → primitive → creates columns. (int, varchar)  
} Non-primitive → cardinality



Rule 1 → for each class in your class diagram → create a table.

Rule 2 → for all primitive attr. (int, string, double, etc) → create columns.

Rule 3 → for all non-primitive attrs → Define cardinality → represent relationship.

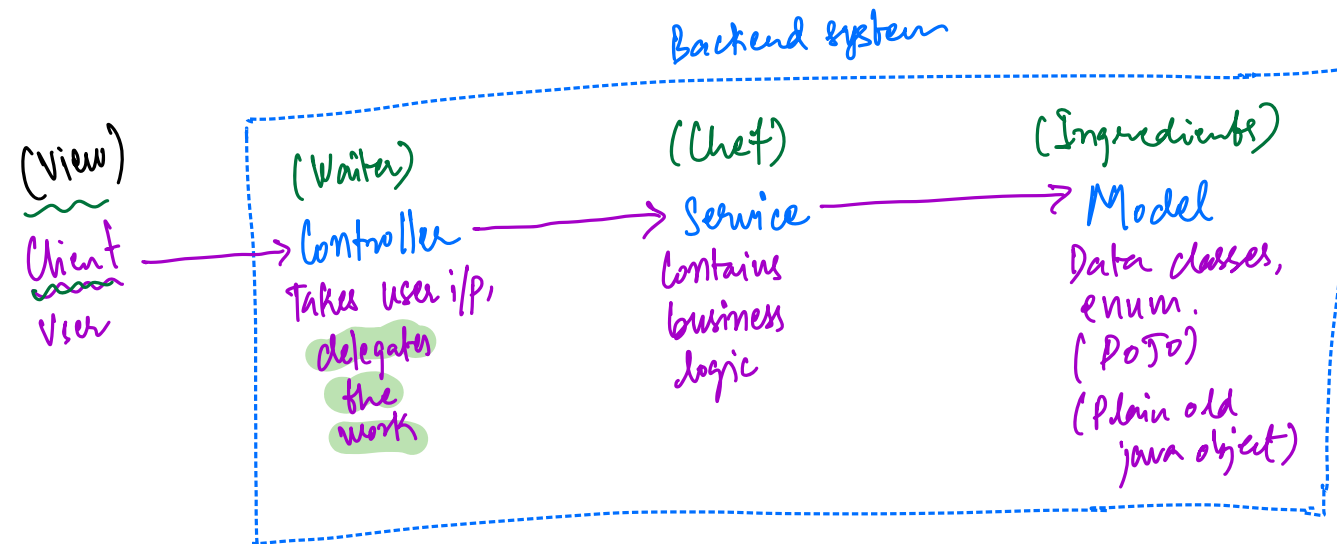


# How to code → Why?  
90 mins to code → Put everything in one giant file X  
→ Clearly organised ✓

MVC Pattern → Model {View} Controller.  
VI

Refrigerator.

Repository X



How to approach this in an interview?

- ① Models first → All your data classes, enums, interfaces. (10 mins)
- ② One requirement → pick the simplest flow and build end to end.
- ③ Next requirement →
- ④ Repeat

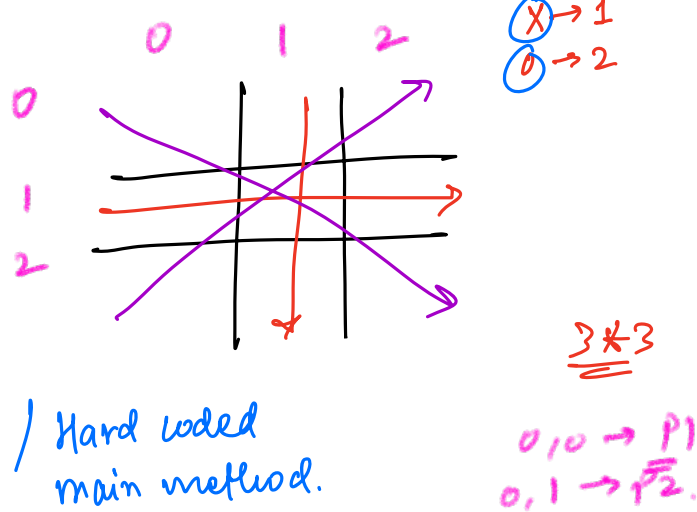
# → Design Tic Tac Toe.

Step 1 → Get an overview

① Entity Design or full system

② Do we need to persist data? → in memory storage

③ How will users interact? → Command line / Hard coded main method.



Step 2:- Gather requirements.

① Board size →  $N \times N$  board;  $N=5$  is max (validations)

② No. of players →  $N-1$  players.

③ Symbols → We need to handle players symbols?  
→ Each player can choose their own symbol before game starts.  
→ No 2 players have the same symbol.

④ Bot players → Yes, we should support bot player  
→ 1 bot per game  
→ Bot can have diff. difficulty levels.  
→ Strategy Pattern.

⑤ Turn Order → Randomize it.  
List  $[P_0, P_1, P_2, P_3, P_4]$   
Start  $\rightarrow [P_1, P_0, P_4, P_3, P_2]$  Randomize

⑥ Winning. → Strategy Pattern  
→ Row Win }  
→ Col . . } → Winning Strategy.  
→ Diag . . }

$3 \times 3 \rightarrow 9$  cells.

2 players

$4 \times 4 \rightarrow 16$  cells.

3 players.

$N^2$  cells.

→  $N-1$  players

→  $\approx N$  cells

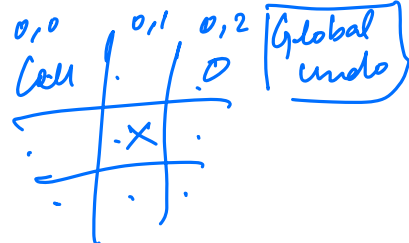
$N-1$  symbols

$\{ X, \_ , \cdot , \cdot \}$   
P1 P2 P3

#, \$, ~

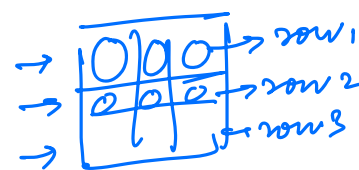
$N=5$

# ⑦ Undo Feature. → Global undo.



Step 3 → Class Diagram

→ Game, Board, Cell, Player, Bot, Symbol, Move



Game

- Board
- list <Player>
- list <Move>
- Winner (Player)
- Game State
- nextPlayerIndex

enum GameState {

- IN-PROGRESS,
- ENDED,
- DRAW

}

Board {

- size: int
- list <list <Cell>>> → 2D grid

}

Cell {

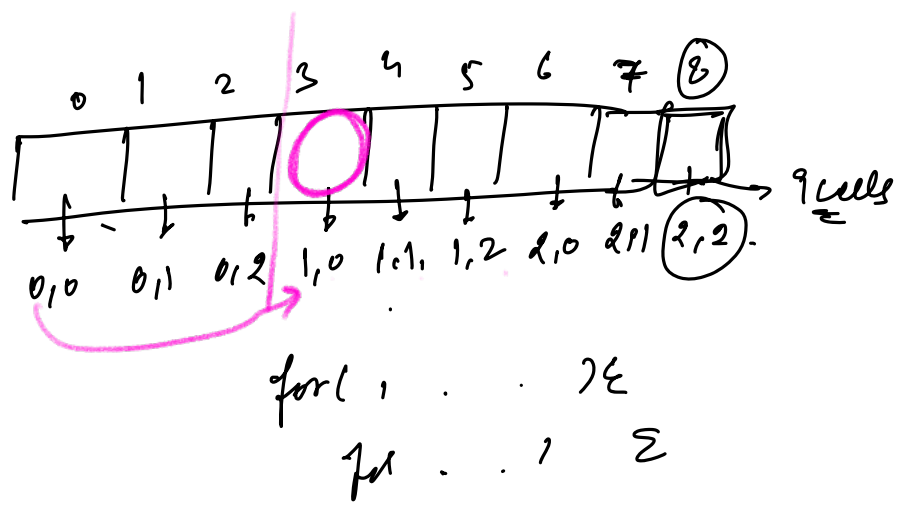
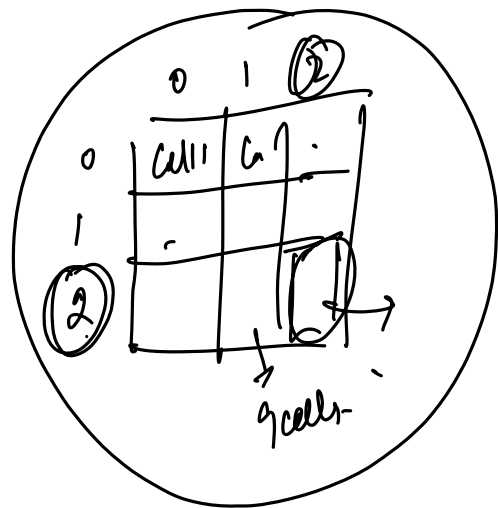
- row: int
- col: int
- CellState
- Player

}

CellState {

- EMPTY,
- FILLED,
- BLOCKED

}



abstract Player {

| - name  
| - symbol : Symbol  
| - playerType  
}

Symbol {

| - char  
| - color  
}

\$

\$

Bot extends Player {

| - botDifficultyLevel  
| - botPlayingStrategy  
}

Move {

| - player  
| - cell  
}

Human extends Player {

| -  
| :  
}

→ Design Patterns?

\* Builder → Game creation

\* Factory → Player Factory

\* Strategy → Bot playing Strategy

\* Strategy → Winning Strategy. → Row, Col, Diagonal.

